Lekcja 12. Odwołania do elementów członkowskich obiektów

Klasa 4 technik programista | Programowanie aplikacji | INF.04

Notatka dotyczy korzystania z pól i metod obiektu w języku C++ przy użyciu operatora kropki. Materiał obejmuje przykłady z klasami Pracownik i Prostokat oraz polem typu strukturalnego Data.

1. Nowe pojęcia i skróty

Pojęcie	Znaczenie
OOP	programowanie obiektowe; technika budowania programu z klas i obiektów.
operator .	operator wyboru / dostępu do elementu członkowskiego obiektu.
pole	zmienna członkowska przechowująca dane obiektu.
metoda	funkcja członkowska wykonywana dla konkretnego obiektu.
member selection operator	angielska nazwa operatora wyboru elementu członkowskiego.
member access operator	angielska nazwa operatora dostępu do elementu członkowskiego.

2. Ogólna postać odwołania

Do elementów członkowskich obiektu odwołujemy się za pomocą operatora kropki. Po lewej stronie kropki zapisujemy nazwę zmiennej obiektowej, a po prawej stronie nazwę pola lub metody należącej do obiektu.

```
zmienna_obiektowa.zmienna_czlonkowska
zmienna_obiektowa.funkcja_czlonkowska();
```

Przykłady z materiału:

```
pracownik.imie
pracownik.nazwisko
pracownik.wyswietlDane();
```

Uwaga

Operator kropki jest także nazywany operatorem dostępu do elementu członkowskiego. Dostęp bezpośredni jest możliwy tylko do elementów widocznych w danym miejscu programu, np. publicznych.

3. Odwołanie do zmiennej członkowskiej

Zmienna członkowska, czyli pole, przechowuje dane obiektu. Instrukcja pracownik.imie = "Jan"; zapisuje wartość w polu imie konkretnego obiektu pracownik. Inny obiekt tej samej klasy może mieć w swoim polu imie zupełnie inną wartość.

4. Odwołanie do funkcji członkowskiej

Funkcję członkowską, czyli metodę, wywołujemy podobnie jak zwykłą funkcję, ale poprzedzamy ją nazwą obiektu i kropką. Nawiasy są obowiązkowe, ponieważ następuje wywołanie funkcji.

```
pracownik.wyswietlDane();
pracownik.ustawImie("Adam");
pracownik.ustawNazwisko("Nowak");
```

5. Przykład 11.3 - klasa Pracownik

W przykładzie zdefiniowano klasę Pracownik, która posiada pola imie i nazwisko oraz metody ustawiające i wyświetlające dane.

```
class Pracownik {
public:
    string imie, nazwisko;

    void ustawImie(string);
    void ustawNazwisko(string);

    void wyswietlDane() {
        cout << "Dane pracownika: " << imie << " " << nazwisko << endl;
    }
};
```

Definicje metod znajdujących się poza klasą zapisuje się z użyciem operatora zakresu ::, np. void Pracownik::ustawImie(string pImie). Sama metoda może zmieniać pole obiektu.

```
void Pracownik::ustawImie(string pImie) {
    imie = pImie;
}

void Pracownik::ustawNazwisko(string pNazwisko) {
    nazwisko = pNazwisko;
}
```

W funkcji main tworzony jest obiekt i wykonywane są odwołania do jego pól i metod:

```
Pracownik pracownik;

pracownik.imie = "Jan";
pracownik.nazwisko = "Kowalski";
pracownik.wyswietlDane();

pracownik.ustawImie("Adam");
pracownik.ustawNazwisko("Nowak");
pracownik.wyswietlDane();
```

Najpierw wartości pól są ustawiane bezpośrednio. Następnie te same wartości są modyfikowane pośrednio przez metody.

6. Przykład 11.4 - klasa Prostokat

Klasa Prostokat pokazuje, że metody mogą wykonywać obliczenia na podstawie danych przechowywanych w polach obiektu.

```
class Prostokat {
public:
    float bok1, bok2;
    float pole();
    float obwod();
};
```

```
float Prostokat::pole() {
    return bok1 * bok2;
}

float Prostokat::obwod() {
    return 2 * bok1 + 2 * bok2;
}
```

```
Prostokat prostokat;
prostokat.bok1 = 1;
prostokat.bok2 = 2;
cout << "Pole wynosi: " << prostokat.pole() << endl;
cout << "Obwod wynosi: " << prostokat.obwod() << endl;
```

7. Przykład 11.5 - pole typu strukturalnego

Element członkowski klasy może być nie tylko typem prostym, ale również typem zdefiniowanym przez programistę. W przykładzie struktura Data jest wykorzystana jako typ pola data_urodzenia.

```
struct Data {
    int dd, mm, rr;
};

class Pracownik {
public:
    int id {};
    string imie {}, nazwisko {};
    Data data_urodzenia {};
};
```

Do elementu zagnieżdżonego można odwołać się przez kolejne użycie operatora kropki:

```
pracownik.data_urodzenia.dd
pracownik.data_urodzenia.mm
pracownik.data_urodzenia.rr
```

Do zapamiętania

Inicjalizacja zerowa {} nadaje polom wartości początkowe. W praktyce za inicjalizację obiektów najczęściej odpowiadają konstruktory, które będą omawiane później.

8. Typowe błędy

Błąd	Poprawnie	Wyjaśnienie
Pracownik.imie	pracownik.imie	Po lewej stronie kropki musi być obiekt, nie nazwa klasy.
pracownik.wyswietlDane	pracownik.wyswietlDane();	Metodę trzeba wywołać z nawiasami.
pracownik.data_urodzenia	pracownik.data_urodzenia.dd	Jeśli potrzebny jest dzień, trzeba wskazać pole wewnątrz struktury.

9. Do zapamiętania

- Do elementów członkowskich obiektu odwołujemy się operatorem kropki.
- Pola przechowują stan obiektu, a metody opisują działania wykonywane na obiekcie.
- Metoda może korzystać z pól tego samego obiektu.

• Elementem członkowskim może być również pole typu strukturalnego lub innego typu złożonego.

• Bezpośredni dostęp do pola wymaga odpowiedniej widoczności elementu.